

# OPEN SOURCE SIEM PROJECT

---

*Complete Implementation Guide*

ELK Stack • Wazuh • Sysmon • Atomic Red Team • TheHive

**Author: Muhammad Usman Saeed**

May 2026 | Version 1.0 | Confidential

# Executive Summary

This document is the complete implementation guide for an enterprise-grade open source SIEM (Security Information and Event Management) system, built from scratch on a personal laptop. It covers every step of the build in the exact order that worked, explains the reasoning behind every architectural decision, and includes all commands, configuration files, and detection results.

The project was designed to demonstrate real SOC analyst and security engineering capability — not just tool familiarity, but deep understanding of how security monitoring works end to end: from raw log collection on an endpoint, through detection, to automated case management and documented incident investigation.

| Metric                            | Result                      |
|-----------------------------------|-----------------------------|
| Total security events collected   | 1,379+ across multiple days |
| MITRE ATT&CK techniques detected  | 11 unique techniques        |
| SIEM dashboards deployed          | 8 Wazuh security dashboards |
| Time to detect simulated attack   | Under 30 seconds            |
| Time to case creation (automated) | Under 60 seconds            |
| Full triage workflow completed    | 8 minutes end to end        |

# System Architecture

The SIEM is built as a layered pipeline. Each layer has a single responsibility. Data flows in one direction — from endpoints inward to storage, then outward to detection and response. This mirrors the architecture of commercial SIEM products like Splunk, IBM QRadar, and Microsoft Sentinel.

| Layer             | Tool                 | Responsibility                              | Network Address | Port(s)         |
|-------------------|----------------------|---------------------------------------------|-----------------|-----------------|
| Endpoint Agent    | Wazuh Agent 4.7.3    | Collects all Windows security events        | Windows host    | outbound        |
| Deep Visibility   | Sysmon v15.20        | Process, registry, network event logging    | Windows service | —               |
| Detection Engine  | Wazuh Manager 4.7.3  | 3000+ rules, MITRE mapping, FIM, compliance | 172.20.0.14     | 1514/1515/55000 |
| Log Processing    | Logstash 8.13.4      | Parse, filter, normalize, enrich            | 172.20.0.11     | 5044/5514       |
| Storage & Search  | Elasticsearch 8.13.4 | Index and store all security events         | 172.20.0.10     | 9200            |
| Visualization     | Kibana 8.13.4        | Analyst dashboards and threat hunting       | 172.20.0.12     | 5601            |
| Case Management   | TheHive 5.2          | Incident tracking and investigation         | 172.20.0.15     | 9000            |
| Attack Simulation | Atomic Red Team      | MITRE ATT&CK technique validation           | Windows host    | local           |

**WHY:** Fixed IPs on a dedicated Docker bridge network (172.20.0.0/24) mean every service always knows where to find every other service. Dynamic IPs would break config files every time a container restarts.

## Data Flow — How a Security Event Becomes an Alert

- Sysmon captures a suspicious process on Windows (e.g., reg.exe spawned by PowerShell)
- Wazuh Agent reads the Sysmon event from the Windows Event Log and ships it to Wazuh Manager on port 1514
- Wazuh Manager applies detection rules, matches MITRE ATT&CK techniques, assigns severity level
- Wazuh Manager ships the alert as JSON to Elasticsearch via Filebeat on port 9200
- Elasticsearch indexes the alert in the wazuh-alerts-4.x-YYYY.MM.DD index
- Kibana reads from Elasticsearch and displays the alert on the security dashboard in real time
- Wazuh integrator detects a level 7+ alert and calls the TheHive integration script
- TheHive creates a case automatically with MITRE tags, severity, and assignee populated
- SOC Analyst opens the case, investigates, documents findings, and closes as resolved

## Why These Tools — and Why Not Others

| Decision           | Chose                           | Rejected                | Reason                                                                                                                  |
|--------------------|---------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Agent management   | Wazuh                           | Elastic Fleet Server    | Wazuh provides 3000+ detection rules out of the box. Fleet Server is just an agent manager with zero detection content. |
| Containerization   | Docker Compose                  | VMs / bare metal        | One command starts the entire stack. Infrastructure-as-code — reproducible and version controllable.                    |
| Docker runtime     | Docker Engine in WSL2           | Docker Desktop          | Docker Desktop adds a GUI layer and extra memory overhead. Engine inside WSL2 is identical to production Linux.         |
| Process visibility | Sysmon + SwiftOnSecurity config | Windows Event Logs only | Standard Windows logs show WHAT happened. Sysmon shows HOW — full process chain, parent, command line.                  |
| Case management    | TheHive 5                       | Manual tracking         | Automated case creation from alerts mirrors real SOC workflow and satisfies compliance documentation requirements.      |
| Attack simulation  | Atomic Red Team                 | Manual testing          | ATT&CK-mapped tests prove detections work. Without validation, detection rules are assumptions not facts.               |

# Phase 1 — Environment Setup

## 1.1 Hardware Specifications

| Specification          | Value                  |
|------------------------|------------------------|
| Machine                | Lenovo E16 Gen 3       |
| Host OS                | Windows 11             |
| Total RAM              | 16 GB                  |
| WSL2 RAM Allocation    | 10 GB                  |
| Free Disk (WSL2)       | 954 GB                 |
| Docker Version         | 29.4.1                 |
| Docker Compose Version | v5.1.3                 |
| Elasticsearch Heap     | 2 GB (50% of WSL2 RAM) |

**WHY:** 16GB RAM allows running the full stack simultaneously without compromise. The rule for Elasticsearch heap: never exceed 50% of available RAM, never exceed 32GB. 2GB heap on 10GB WSL2 allocation follows this exactly.

## 1.2 Install WSL2 with Ubuntu

Open Windows PowerShell as Administrator:

```
wsl --install
```

Restart your machine. Ubuntu 22.04 opens automatically. Create a username and password when prompted.

**IMPORTANT:** Write down your Ubuntu username and password. You will need them every session.

## 1.3 Configure WSL2 Memory Limit

Create C:\Users\YourName\.wslconfig with these contents:

```
[wsl2]
memory=10GB
processors=6
swap=4GB
```

Restart WSL2:

```
wsl --shutdown
```

**WHY:** Without this file, WSL2 grows dynamically and can consume all 16GB RAM, freezing Windows. The cap gives 10GB to the SIEM stack and leaves 6GB for Windows to stay responsive.

## 1.4 Install Docker Engine Inside WSL2

Open Ubuntu terminal. Run these commands in sequence:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg lsb-release
```

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
sudo usermod -aG docker $USER
sudo service docker start
```

Close and reopen Ubuntu terminal, then verify:

```
docker --version          # Should show 29.4.1
docker compose version    # Should show v5.1.3
docker run hello-world    # Should print Hello from Docker
```

## 1.5 Set Critical Elasticsearch Kernel Parameter

**IMPORTANT:** This must be done before starting Elasticsearch or it will exit immediately with no clear error message.

```
sudo sysctl -w vm.max_map_count=262144
echo 'vm.max_map_count=262144' | sudo tee -a /etc/sysctl.conf
```

**WHY:** Elasticsearch uses memory-mapped files for index storage. Linux's default limit of 65530 virtual memory areas is too low. 262144 is Elasticsearch's documented minimum. This setting persists across WSL2 restarts via /etc/sysctl.conf.

## 1.6 Create Project Directory Structure

```
mkdir -p ~/siem-lab/{logstash/{config,pipeline},certs,wazuh}
cd ~/siem-lab
```

## Phase 2 — Deploy the ELK Stack

The entire stack is defined in one `docker-compose.yml` file. This is infrastructure-as-code — the whole SIEM starts with a single command and is fully reproducible.

### 2.1 Create `docker-compose.yml`

```
nano ~/siem-lab/docker-compose.yml
```

Paste the complete file below. Do not modify any indentation — YAML is whitespace sensitive.

```
# — NETWORK — private bridge, all containers talk here —
networks:
  siem-net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/24
```

```
# — VOLUMES — data survives container restarts —————
volumes:
  elasticsearch-data:
  kibana-data:
  wazuh-manager-etc:
  wazuh-manager-var:
  thehive-data:
  thehive-index:
  thehive-db:
```

```
services:
```

```
# — ELASTICSEARCH —————
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.13.4
  container_name: elasticsearch
  environment:
    - node.name=es-node-1
    - cluster.name=siem-cluster
    - discovery.type=single-node
    - ELASTIC_PASSWORD=SiemLab@2025!
    - bootstrap.memory_lock=true
    - xpack.security.enabled=true
    - xpack.security.http.ssl.enabled=false
    - xpack.security.transport.ssl.enabled=false
    - ES_JAVA_OPTS=-Xms2g -Xmx2g
  ulimits:
    memlock:
      soft: -1
      hard: -1
  volumes:
    - elasticsearch-data:/usr/share/elasticsearch/data
  ports:
    - "9200:9200"
  networks:
    siem-net:
      ipv4_address: 172.20.0.10
  healthcheck:
    test: ["CMD-SHELL", "curl -s -u elastic:SiemLab@2025! http://localhost:9200/_cluster/health | grep -q 'green\\|yellow'"]
    interval: 30s
    timeout: 10s
```

```
retries: 10
```

```
# — LOGSTASH —————
logstash:
  image: docker.elastic.co/logstash/logstash:8.13.4
  container_name: logstash
  environment:
    - LS_JAVA_OPTS=-Xms512m -Xmx512m
    - ELASTIC_PASSWORD=SiemLab@2025!
  volumes:
    - ./logstash/config/logstash.yml:/usr/share/logstash/config/logstash.yml:ro
    - ./logstash/pipeline:/usr/share/logstash/pipeline:ro
  ports:
    - "5044:5044"
    - "5514:5514/udp"
    - "9600:9600"
  networks:
    siem-net:
      ipv4_address: 172.20.0.11
  depends_on:
    elasticsearch:
      condition: service_healthy
```

```
# — KIBANA —————
kibana:
  image: docker.elastic.co/kibana/kibana:8.13.4
  container_name: kibana
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    - ELASTICSEARCH_USERNAME=kibana_system
    - ELASTICSEARCH_PASSWORD=SiemLab@2025!
    - XPACK_SECURITY_ENCRYPTIONKEY=a7f3c9d2e1b4a6f8c0d2e4f6a8b0c2d4
    - XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY=b8e3d0clf4a7b2e5d8clf4a7b2e5d8c1
    - XPACK_REPORTING_ENCRYPTIONKEY=c9f4e1d2b5a8c3f6e1d4b7a0c3f6e9d2
  volumes:
    - kibana-data:/usr/share/kibana/data
  ports:
    - "5601:5601"
  networks:
    siem-net:
      ipv4_address: 172.20.0.12
  depends_on:
    elasticsearch:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "curl -s http://localhost:5601/api/status | grep -q 'available'"]
    interval: 30s
    timeout: 10s
    retries: 10
```

```
# — WAZUH MANAGER —————
wazuh-manager:
  image: wazuh/wazuh-manager:4.7.3
  container_name: wazuh-manager
  hostname: wazuh-manager
  restart: unless-stopped
  environment:
    - INDEXER_URL=http://elasticsearch:9200
    - INDEXER_USERNAME=elastic
    - INDEXER_PASSWORD=SiemLab@2025!
    - FILEBEAT_SSL_VERIFICATION_MODE=none
  volumes:
    - wazuh-manager-etc:/var/ossec/etc
```

```

- wazuh-manager-var:/var/ossec/var
ports:
- "1514:1514"
- "1514:1514/udp"
- "1515:1515"
- "55000:55000"
networks:
  siem-net:
    ipv4_address: 172.20.0.14
depends_on:
  elasticsearch:
    condition: service_healthy

```

```

# --- THEHIVE ---
thehive:
  image: strangebee/thehive:5.2
  container_name: thehive
  hostname: thehive
  restart: unless-stopped
  mem_limit: 1500m
  environment:
    - JVM_OPTS=-Xms512m -Xmx1g
  volumes:
    - thehive-data:/opt/thehive/data
    - thehive-index:/opt/thehive/index
    - thehive-db:/opt/thehive/db
  ports:
    - "9000:9000"
  networks:
    siem-net:
      ipv4_address: 172.20.0.15
  depends_on:
    elasticsearch:
      condition: service_healthy

```

**NOTE:** TheHive requires three volumes (data, index, db) to persist user accounts and cases across restarts. Missing thehive-db causes users to be wiped on every container restart.

## 2.2 Create Logstash Configuration Files

Create the Logstash settings file:

```
nano ~/siem-lab/logstash/config/logstash.yml
```

Contents:

```

http.host: "0.0.0.0"
xpack.monitoring.enabled: false

```

Create the processing pipeline:

```
nano ~/siem-lab/logstash/pipeline/main.conf
```

Contents:

```

input {
  beats { port => 5044 }
  udp { port => 5514
    codec => plain
    tags => ["syslog"] }
}
filter {
  if "syslog" in [tags] {

```

```

    grok { match => { "message" => "%{SYSLOGTIMESTAMP:syslog_timestamp}
%{SYSLOGHOST:syslog_host} %{DATA:syslog_program} (?:\[ %{POSINT:syslog_pid} \])?:
%{GREEDYDATA:syslog_message}" } }
    date { match => [ "syslog_timestamp", "MMM d HH:mm:ss", "MMM dd HH:mm:ss" ] }
  }
  mutate { add_field => { "[@metadata][index_prefix]" => "siem-logs" } }
}
output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    user => "elastic"
    password => "SiemLab@2025!"
    index => "%{[@metadata][index_prefix]}-%{+YYYY.MM.dd}"
  }
}

```

**WHY:** The grok filter is log normalization — it converts raw unstructured text into named searchable fields. Without normalization, you cannot write detection rules or build dashboards based on specific field values.

## 2.3 Start the Stack

```

cd ~/siem-lab
docker compose up -d

```

Watch containers reach healthy status (takes 2-3 minutes):

```
watch docker compose ps
```

Once Elasticsearch is healthy, set the Kibana system password:

```

curl -s -X POST -u elastic:SiemLab@2025! -H "Content-Type: application/json"
http://localhost:9200/_security/user/kibana_system/_password -d
'{"password":"SiemLab@2025!"}'

```

Expected response: {} (empty braces = success)

Open Windows browser: <http://localhost:5601> | Login: elastic / SiemLab@2025!

## 2.4 Verify Stack Health

```

curl -s -u elastic:SiemLab@2025! http://localhost:9200/_cluster/health | python3 -m
json.tool

```

Status should be green or yellow. Red means Elasticsearch is not fully started yet — wait 60 seconds and retry.

## Phase 3 — Connect Windows Endpoint to SIEM

The Wazuh Agent on Windows is what feeds real security data into the SIEM. Without an agent, the SIEM has no endpoint telemetry to analyze.

### 3.1 Set Up Windows-to-WSL2 Port Forwarding

**IMPORTANT:** WSL2 gets a new internal IP every time Windows restarts. This port proxy command must be run at the start of every session.

Open PowerShell as Administrator:

```
$wslIP = (wsl hostname -I).Trim().Split()[0]
Write-Host "WSL2 IP is: $wslIP"
netsh interface portproxy delete v4tov4 listenport=1514 listenaddress=0.0.0.0
netsh interface portproxy delete v4tov4 listenport=1515 listenaddress=0.0.0.0
netsh interface portproxy add v4tov4 listenport=1514 listenaddress=0.0.0.0
connectport=1514 connectaddress=$wslIP
netsh interface portproxy add v4tov4 listenport=1515 listenaddress=0.0.0.0
connectport=1515 connectaddress=$wslIP
netsh interface portproxy show all
```

**WHY:** WSL2 runs in its own network namespace. Windows cannot directly reach container ports without forwarding. Port proxy intercepts traffic on Windows loopback (127.0.0.1) and forwards to WSL2 — the same concept as NAT on a router or AWS security group port forwarding.

**NOTE:** Port 1514 TCP is for agent data communication. Port 1514 UDP is for syslog sources. Port 1515 TCP is for agent registration. All three must be exposed in docker-compose.yml and forwarded via port proxy.

### 3.2 Install Wazuh Agent on Windows

Run in PowerShell as Administrator:

```
Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.3-1.msi"
-OutFile "C:\wazuh-agent.msi"
msiexec /i C:\wazuh-agent.msi /q WAZUH_MANAGER="127.0.0.1"
WAZUH_REGISTRATION_SERVER="127.0.0.1" WAZUH_AGENT_NAME="windows-host"
NET START WazuhSvc
```

Verify agent registered and is Active (run in Ubuntu):

```
docker exec wazuh-manager /var/ossec/bin/agent_control -l
```

Expected: ID: 00X, Name: windows-host, IP: any, Active

**IMPORTANT:** If agent shows Never Connected — the port proxy is pointing to a stale WSL2 IP. Re-run Step 3.1 with the fresh IP and restart the agent service.

### 3.3 Install Sysmon for Deep Endpoint Visibility

Run in PowerShell as Administrator:

```
Invoke-WebRequest -Uri "https://download.sysinternals.com/files/Sysmon.zip" -OutFile
"C:\Sysmon.zip"
Expand-Archive -Path "C:\Sysmon.zip" -DestinationPath "C:\Sysmon" -Force
Invoke-WebRequest -Uri "https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-
config/master/sysmonconfig-export.xml" -OutFile "C:\Sysmon\sysmonconfig.xml"
cd C:\Sysmon
.\Sysmon64.exe -accepteula -i sysmonconfig.xml
```

**WHY:** Standard Windows Event Logs show logons and service changes. Sysmon adds full process creation chains, network connections, and registry modifications — the data needed to

detect living-off-the-land attacks. Without Sysmon we detected 1 MITRE technique. With Sysmon we detected 11.

**NOTE:** We use the SwiftOnSecurity config — the industry standard Sysmon baseline used in real enterprise environments. It minimizes noise while capturing all meaningful security events.

### 3.4 Connect Sysmon to Wazuh Agent

Run in PowerShell as Administrator — adds Sysmon log channel to Wazuh agent config:

```
$configPath = "C:\Program Files (x86)\ossec-agent\ossec.conf"
$sysmonConfig = "`n  <localfile>`n    <location>Microsoft-Windows-
Sysmon/Operational</location>`n    <log_format>eventchannel</log_format>`n
</localfile>"
$content = Get-Content $configPath -Raw
$content = $content -replace '</ossec_config>', "$sysmonConfig`n</ossec_config>"
Set-Content $configPath $content
NET STOP WazuhSvc
NET START WazuhSvc
```

**WHY:** Without this config block, Wazuh Agent ignores Sysmon events completely even though Sysmon is running and logging. This one change is the difference between basic and deep endpoint detection coverage.

## Phase 4 — Load Security Dashboards

Wazuh provides 8 pre-built security dashboards for Kibana covering threat detection, compliance, malware, and incident response.

### 4.1 Import Dashboards

```
wget -O /tmp/wazuh-dashboards.ndjson
https://packages.wazuh.com/integrations/elastic/4.x-8.x/dashboards/wz-es-4.x-8.x-
dashboards.ndjson
curl -s -X POST "http://localhost:5601/api/saved_objects/_import?overwrite=true" \
  -u elastic:SiemLab@2025! \
  -H "kbn-xsrf: true" \
  --form file=@/tmp/wazuh-dashboards.ndjson
```

Success shows 8 dashboards imported including wazuh-security-events-v1.0, wazuh-vulnerabilities-v1.0, wazuh-malware-detection-v1.0, wazuh-pci-dss-v1.0, and wazuh-incident-response-v1.0.

### 4.2 Create Kibana Data View

In Kibana: Stack Management → Index Patterns → Create index pattern

- Name: wazuh-alerts-4.x-\*
- Timestamp field: @timestamp
- Click Create index pattern

Go to Discover → select wazuh-alerts-4.x-\* → set Last 7 days. Security events will appear immediately.

### 4.3 Open Main Dashboard

Dashboard → wazuh-security-events-v1.0 → Last 7 days

You will see: total alert count, alerts-over-time chart, MITRE ATT&CK tactics pie chart, and a security alerts table with technique IDs, tactics, rule descriptions, and severity levels.

## Phase 5 — Attack Simulation and Detection Validation

Atomic Red Team is a library of MITRE ATT&CK-mapped attack simulations. Running these tests proves that our SIEM actually detects the attacks it is supposed to detect — without this step, detection rules are unvalidated assumptions.

### 5.1 Install Atomic Red Team

Run in PowerShell as Administrator:

```
Set-ExecutionPolicy Bypass -Scope Process -Force
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)
Install-AtomicRedTeam -getAtomics -Force
```

**NOTE:** Set-ExecutionPolicy Bypass -Scope Process -Force must be run at the start of every new PowerShell session before using Atomic Red Team. It only lasts for that session.

### 5.2 Run Attack Simulations

Import the module and run each test:

```
Import-Module "C:\AtomicRedTeam\invoke-atomicredteam\Invoke-AtomicRedTeam.psdl" -Force
Invoke-AtomicTest T1082 -TestNumbers 1 # System Information Discovery
Invoke-AtomicTest T1033 -TestNumbers 1 # User and Session Discovery
Invoke-AtomicTest T1016 -TestNumbers 1 # Network Configuration Discovery
Invoke-AtomicTest T1069.001 -TestNumbers 1 # Local Group Discovery
Invoke-AtomicTest T1087.001 -TestNumbers 1 # Local Account Discovery
Invoke-AtomicTest T1547.001 -TestNumbers 1 # Registry Run Key Persistence
```

| Technique        | What It Simulates                                             | Why Attackers Do This                                                 |
|------------------|---------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>T1082</b>     | System information discovery — OS version, hardware, software | Map the target before attacking — understand what you're dealing with |
| <b>T1033</b>     | User and session enumeration — who is logged in               | Find high-value targets — are admins currently active?                |
| <b>T1016</b>     | Network configuration — IPs, subnets, routes                  | Map the internal network to find other systems to move to             |
| <b>T1069.001</b> | Local group membership — who is in Administrators             | Identify lateral movement targets with elevated privileges            |
| <b>T1087.001</b> | Local account enumeration — all user accounts                 | Find accounts to target for credential attacks                        |
| <b>T1547.001</b> | Registry Run Key — writes payload to startup location         | Ensure payload survives reboots — persistence mechanism               |

### 5.3 Verify Detections

Run this in Ubuntu after tests complete — shows every MITRE technique detected:

```
docker exec wazuh-manager tail -500 /var/ossec/logs/alerts/alerts.json | python3 -c "
import sys, json
techniques = {}
for line in sys.stdin:
    line = line.strip()
    if not line: continue
```

```
try:
    a = json.loads(line)
    mitre = a.get('rule', {}).get('mitre', {})
    for tid in mitre.get('id', []):
        if tid not in techniques:
            techniques[tid] = {'desc': a.get('rule', {}).get('description', ''),
'level': a.get('rule', {}).get('level', 0)}
        except: pass
for tid, info in sorted(techniques.items()):
    print(f'{tid} | Level {info["level"]} | {info["desc"]}')
"
```

## Phase 6 — Detection Results

After enabling Sysmon and running atomic tests, the SIEM detected 11 unique MITRE ATT&CK techniques. Below is every detection with its meaning.

| Technique | Level | Tactic            | Detection Description                                          |
|-----------|-------|-------------------|----------------------------------------------------------------|
| T1027     | 10    | Defense Evasion   | Base64 encoded data in registry key — fileless malware storage |
| T1112     | 10    | Defense Evasion   | Registry modified with encoded content — obfuscation technique |
| T1105     | 9     | Command & Control | PowerShell created executable in Windows root — C2 staging     |
| T1547.001 | 6     | Persistence       | Registry Run Key modified via reg.exe — auto-start persistence |
| T1059.001 | 4     | Execution         | PowerShell launched from suspicious parent process             |
| T1059.003 | 4     | Execution         | CMD started by abnormal process — living off the land          |
| T1574.001 | 3     | Multiple          | net.exe spawned by CMD — built-in tool abuse                   |
| T1087     | 3     | Discovery         | Account discovery commands executed                            |
| T1033     | 3     | Discovery         | User and session enumeration                                   |
| T1078     | 3     | Multiple          | Valid account logon — valuable in correlation context          |
| T1562.001 | 3     | Defense Evasion   | Wazuh agent stopped — SIEM detected its own tampering          |

### Key Detections Explained for Interviews

#### T1027 and T1112 — Level 10 — Highest Priority

Attackers store Base64-encoded payloads in registry keys to hide from file-based antivirus. PowerShell decodes and executes the payload in memory — no file ever touches disk. This is fileless malware. Level 10 means near-certain malicious activity.

#### T1547.001 — Level 6 — Persistence

Our test wrote to HKCU\Software\Microsoft\Windows\CurrentVersion\Run. Any value here executes automatically at every user logon. This is how ransomware and RATs survive reboots — still used in 2026 because Windows still supports it.

#### T1562.001 — Agent Tampering Detection

Wazuh logged when its own agent service was stopped. Attackers kill security tools immediately after gaining access. The alert was already shipped to Elasticsearch before the agent went down — it cannot be deleted from the endpoint after the fact.

#### T1078 — Correlation is Everything

Every Windows logon fires T1078. Alone it is low-severity noise. Combined with T1105 and T1547.001 from the same host in a 5-minute window, it signals an active compromise with the attacker using valid credentials after establishing a foothold.

# Phase 7 — TheHive Case Management

TheHive converts raw SIEM alerts into tracked, documented incidents. This is the operational layer of the SOC — without it, alerts fire and disappear. With it, every alert becomes an accountable investigation with timeline, findings, and resolution.

**WHY:** Every compliance framework — PCI-DSS, ISO 27001, SOC 2, HIPAA — requires documented incident response. TheHive provides that documentation automatically. It is not just a nice-to-have; it is how you demonstrate compliance to auditors.

## 7.1 Initial TheHive Setup

Open `http://localhost:9000` — Login: `admin@thehive.local` / `secret`

Create SOC-Lab organisation: Admin → Organisations → Create Organisation → Name: SOC-Lab

Create analyst user: Admin → Users → Create User:

- Login: `analyst@soclab.local`
- Name: SOC Analyst
- Organisation: SOC-Lab
- Profile: analyst
- Password: `Analyst@2025!`

Get API key: click `analyst@soclab.local` → Create API Key → copy and save immediately.

**IMPORTANT:** TheHive requires three volumes to persist data across restarts: `thehive-data`, `thehive-index`, `thehive-db`. All three must be in `docker-compose.yml`. Missing `thehive-db` wipes users and cases on every restart.

## 7.2 Deploy Wazuh-TheHive Integration Script

Run in Ubuntu — writes the integration script to a temp file then copies into the container:

```
cat > /tmp/custom-thehive.py << 'EOF'
#!/usr/bin/env python3
import sys, json, urllib.request, urllib.error

THEHIVE_URL = "http://172.20.0.15:9000"
THEHIVE_API_KEY = "YOUR_API_KEY_HERE"
SEVERITY_MAP = {1:1,2:1,3:1,4:1,5:1,6:1,7:2,8:2,9:2,10:2,11:3,12:3,13:3,14:3,15:3}

def create_case(alert):
    rule = alert.get('rule', {})
    agent = alert.get('agent', {})
    level = int(rule.get('level', 0))
    if level < 7: sys.exit(0)
    mitre = rule.get('mitre', {})
    ids = mitre.get('id', [])
    tactics = mitre.get('tactic', [])
    title = '[Wazuh] ' + rule.get('description','Alert') + ' - ' +
agent.get('name','unknown')
    desc = 'Agent: ' + agent.get('name','unknown') + '\nLevel: ' + str(level) +
'\nMITRE: ' + str(ids) + '\nTactic: ' + str(tactics)
    case =
{'title':title,'description':desc,'severity':SEVERITY_MAP.get(level,1),'tags':['wazuh']}
+ids,'tlp':2,'status':'New'}
    data = json.dumps(case).encode()
```

```

req = urllib.request.Request(THEHIVE_URL+'/api/v1/case',data=data,
    headers={'Content-Type':'application/json','Authorization':'Bearer '+THEHIVE_API_KEY},method='POST')
try:
    with urllib.request.urlopen(req,timeout=10) as r: print('OK: '+r.read().decode())
except urllib.error.HTTPError as e: print('ERR: '+str(e.code)); sys.exit(1)

if __name__ == '__main__':
    try:
        with open(sys.argv[1]) as f: alert = json.load(f)
        create_case(alert)
    except Exception as e: print('Error: '+str(e)); sys.exit(1)
EOF

```

```

docker cp /tmp/custom-thehive.py wazuh-manager:/var/ossec/integrations/custom-thehive
docker exec wazuh-manager chmod 755 /var/ossec/integrations/custom-thehive
docker exec wazuh-manager chown root:wazuh /var/ossec/integrations/custom-thehive

```

**NOTE:** Replace YOUR\_API\_KEY\_HERE with the actual key from TheHive before copying. The script creates cases only for alerts level 7 and above — this filters out noise and ensures only meaningful events become cases.

## 7.3 Register Integration in Wazuh Config

Verify ossec.conf ends correctly before adding — the integration must be inside ossec\_config tags:

```
docker exec wazuh-manager tail -5 /var/ossec/etc/ossec.conf
```

The last line must be </ossec\_config>. If the integration block is missing, add it with Python to avoid shell escaping issues:

```

docker exec wazuh-manager bash -c "
python3 -c \"
content = open('/var/ossec/etc/ossec.conf').read()
idx = content.rfind('</ossec_config>')
block = '\"\\n <integration>\\n    <name>custom-thehive</name>\\n    <level>7</level>\\n
<alert_format>json</alert_format>\\n  </integration>\\n</ossec_config>\"'
content = content[:idx] + block
open('/var/ossec/etc/ossec.conf', 'w').write(content)
print('Done')
\""

```

```

docker exec wazuh-manager /var/ossec/bin/wazuh-control restart
sleep 20
docker exec wazuh-manager ps aux | grep integrat

```

Expected: wazuh-integratord process running with a PID number.

## 7.4 Test the Integration

Manually test against a real alert to confirm TheHive connection works:

```

docker exec wazuh-manager bash -c "tail -500 /var/ossec/logs/alerts/alerts.json |
python3 -c \"
import sys,json
for line in sys.stdin:
    line=line.strip()
    if not line: continue
    try:

```

```
a=json.loads(line)
if int(a.get('rule',{}).get('level',0))>=7:
    with open('/tmp/test-alert.json','w') as f: json.dump(a,f)
    print('Found level:',a.get('rule',{}).get('level')); break
except: pass
\ ""
```

```
docker exec wazuh-manager python3 /var/ossec/integrations/custom-thehive /tmp/test-
alert.json
```

Expected: OK: {case JSON with \_id, title, severity, assignee}

Trigger automatic cases by running atomic tests from Windows PowerShell:

```
Set-ExecutionPolicy Bypass -Scope Process -Force
Import-Module "C:\AtomicRedTeam\invoke-atomicredteam\Invoke-AtomicRedTeam.psd1" -Force
Invoke-AtomicTest T1547.001 -TestNumbers 1
```

Cases appear in TheHive within 60 seconds automatically.

# Phase 8 — SOC Investigation Workflow

This is the workflow you demonstrate in an interview. It shows you understand the operational side of a SOC — not just detection, but investigation, documentation, and resolution.

## 8.1 The Investigation — Case Study

Scenario: Alert fires. Wazuh Rule 92041, Level 10. T1027 and T1112 — Base64 pattern in registry key from windows-host. TheHive auto-created Case #3. You are the analyst on duty.

### Step 1 — Pull Raw Evidence from SIEM

```
docker exec wazuh-manager tail -500 /var/ossec/logs/alerts/alerts.json | python3 -c "
import sys, json
for line in sys.stdin:
    line = line.strip()
    if not line: continue
    try:
        a = json.loads(line)
        if 'T1027' in a.get('rule', {}).get('mitre', {}).get('id', []):
            rule=a.get('rule', {}); agent=a.get('agent', {});
win=a.get('data', {}).get('win', {})
            print('Timestamp:', a.get('timestamp', ''))
            print('Agent:', agent.get('name', ''), agent.get('ip', ''))
            print('Rule:', rule.get('id', ''), '| Level:', rule.get('level', ''))
            print('Process:', win.get('eventdata', {}).get('image', 'N/A'))
            print('MITRE:', rule.get('mitre', {}).get('id', []))
            break
    except: pass
"
```

### Step 2 — Open Case in TheHive

Login as analyst@soclab.local → Cases → Click the relevant case.

Add three tasks by clicking + next to Tasks:

- Initial Triage — assign to SOC Analyst
- Host Investigation — assign to SOC Analyst
- Containment Decision — assign to SOC Analyst

### Step 3 — Add Observables

Click Observables → + → Add:

- Type: hostname → Value: windows-host
- Type: other → Value: T1027 → Tag: mitre-technique
- Type: other → Value: T1112 → Tag: mitre-technique

### Step 4 — Document Investigation in Tasks

Click Initial Triage → Start → Add log with evidence from Step 1.

Click Host Investigation → Start → Document process lineage and host status.

Click Containment Decision → Start → Document determination and action taken.

Mark all tasks Completed → Change case Status to Resolved → Set resolution: True Positive.

### Step 5 — Close Case

Status: Resolved | Resolution: True Positive — Authorized Simulation

TheHive automatically records Time to Acknowledge and Time to Triage metrics.

# Daily Startup and Shutdown Procedure

Follow this exact sequence every time you start a session. Order matters.

## Startup — Run in This Order

### 1. Start Docker and the SIEM stack (Ubuntu terminal)

```
sudo service docker start
cd ~/siem-lab
docker compose up -d
watch docker compose ps      # Wait until all show healthy or running
```

### 2. Update port proxy with current WSL2 IP (PowerShell as Administrator)

```
netsh interface portproxy delete v4tov4 listenport=1514 listenaddress=0.0.0.0
netsh interface portproxy delete v4tov4 listenport=1515 listenaddress=0.0.0.0
$wslIP = (wsl hostname -I).Trim().Split()[0]
netsh interface portproxy add v4tov4 listenport=1514 listenaddress=0.0.0.0
connectport=1514 connectaddress=$wslIP
netsh interface portproxy add v4tov4 listenport=1515 listenaddress=0.0.0.0
connectport=1515 connectaddress=$wslIP
```

### 3. Start Wazuh Agent (PowerShell as Administrator)

```
NET START WazuhSvc
```

### 4. Verify agent is Active (Ubuntu terminal)

```
docker exec wazuh-manager /var/ossec/bin/agent_control -l
```

### 5. Open tools in browser

- Kibana: <http://localhost:5601> | elastic / SiemLab@2025!
- TheHive: <http://localhost:9000> | analyst@soclab.local / Analyst@2025!

## Shutdown — Before Turning Off Laptop

```
cd ~/siem-lab
docker compose down
```

**IMPORTANT:** Always run docker compose down before shutdown. This preserves all data in Docker volumes cleanly. Closing the laptop without stopping containers can cause Elasticsearch index corruption.

## Re-deploy Integration Script After Restart

The Wazuh integration script lives inside the container and is lost on container restart. After every docker compose down / up, redeploy it:

```
docker cp /tmp/custom-thehive.py wazuh-manager:/var/ossec/integrations/custom-thehive
docker exec wazuh-manager chmod 755 /var/ossec/integrations/custom-thehive
docker exec wazuh-manager chown root:wazuh /var/ossec/integrations/custom-thehive
docker exec wazuh-manager /var/ossec/bin/wazuh-control restart
```

**NOTE:** To make the integration script persist across restarts permanently, add it to the wazuh-manager-var volume by mounting it in docker-compose.yml as a file volume. This is a future improvement.

## Current Status

| Capability                            | Status  | Evidence                                       |
|---------------------------------------|---------|------------------------------------------------|
| Elasticsearch storing security events | WORKING | 1,379+ documents in daily indices              |
| Logstash processing pipeline          | WORKING | Accepting Beats on 5044, Syslog on 5514        |
| Kibana analyst dashboards             | WORKING | 8 Wazuh dashboards loaded with live data       |
| Wazuh Manager detection engine        | WORKING | 3000+ rules active, connected to Elasticsearch |
| Windows endpoint agent                | WORKING | windows-host showing Active, shipping events   |
| Sysmon deep process visibility        | WORKING | v15.20 with SwiftOnSecurity config             |
| MITRE ATT&CK detection coverage       | WORKING | 11 unique techniques validated                 |
| Attack simulation and validation      | WORKING | Atomic Red Team tests confirmed detections     |
| TheHive case management               | WORKING | Auto-creates cases from level 7+ alerts        |
| End-to-end investigation workflow     | WORKING | Full case investigated and resolved in 8 min   |

## Roadmap — Next Phases

### Shuffle SOAR — Automated Playbooks

Shuffle is an open source SOAR platform. Connect it to Wazuh alerts via webhook to automatically trigger response actions — isolate an endpoint, block an IP, send notifications, create TheHive cases. This is the automation layer that makes a SOC scale beyond manual triage.

### MISP — Threat Intelligence Enrichment

MISP is an open source threat intelligence platform. Integrating it with Logstash enriches every incoming log with known bad IPs, domains, and file hashes from global threat feeds. A log event matching a known C2 server automatically becomes a critical alert instead of noise.

### Expand Detection Coverage

Add more Atomic Red Team tests covering credential access (T1003 family), lateral movement, and data exfiltration. Each validated detection adds to the portfolio and provides concrete examples for technical interviews.

### Production Hardening

- Enable mutual TLS on all inter-service communication
- Deploy Elasticsearch as a 3-node cluster for high availability
- Implement Index Lifecycle Management (ILM) for log retention — hot 7 days, warm 30 days, cold 1 year
- Add Kafka between log shippers and Logstash for buffering during high-volume events

- Role-based access control in Kibana — analysts see only their relevant indices